

AIPL の型健全性・型安全性・デッドロック自由（第 3 版）

— 待ちを取り除かずにデッドロック自由を証明する。AIPL⁻² の機械検証 —

児玉工房 / AICE・AIPL プロジェクト

2026 年 08 月 23 日

概要

第 1 版（2026 年 7 月 28 日）は、`reply` と戻り値型を結んだ部分言語 AIPL⁻ について型健全性と型安全性を Coq で機械検証した。ただしデッドロック自由は `await` を言語から取り除いた断片についてしか言えていなかった。待てる構文を捨てて「待ちが返らない」を消していたのだから、当然である。

第 2 版（同年 7 月 30 日）は、期限を必須にすることで進行定理を二択にした。ただしこれは紙の上の証明であり、しかも得られる性質はデッドロックが有限時間の失敗に変わることであって、詰まらないことではない。互いに待ち合う二者は両方が期限切れになる。

本稿（第 3 版）は待ちを取り除かず、期限にも頼らず、デッドロック自由そのものを機械検証する。道具は義務レベルである — `future` の型に「それを埋める側のレベル」を持たせ、待ちはずレベルの高い方へ向かうことを型で要求する。加えて実行時の不変条件を一つ置く — 未解決の `future` には必ずそれを埋める者がいる。この二つから、全タスクが待ち状態になることはない。

主結果は進行定理から枝が一つ消えることである。

進行定理の主張

第 1 版	終状態 $\vee$ 一歩進める $\vee$ 全タスクが待ち
-------	---------------------------------

第 3 版	終状態 $\vee$ 一歩進める
-------	------------------

併せて効果も形式化に入れた。`future` の型にレベルと同じやり方で効果を載せ、送信は引き継がず待ちが引き継ぐという実装どおりの規則を与えると、効果の健全性が保存定理から出る — メソッドが宣言した効果は、`now` で呼んだ先も含めて、そのメソッドが実際に行うことを覆っている。

定理は十三あり、いずれも `Print Assumptions` が Closed under the global context（公理なし）である。併せて、三つの性質（健全性・安全性・デッドロック自由）が同時に成り立つ最大の構文を機構ごとに調べ、`select・timeout・any`／多相・`become` を入れると何が壊れるかを明記する。

定理が空虚でないことも確かめた。`await` と `while` を実際に使うプログラムを形式化し、仮定をすべて満たすことを示し、併せてレベルを揃えると本体の型検査が通らないことを確かめた — 条件が実際に効いていることの対照である。

目次

第 I 部	位置づけ	2
1	三つの版の関係	3
2	何を証明したいのか	3

第 II 部	定式化	3
3	構文	4
3.1	糖衣	4
4	表と構成	4
5	型付け	5
6	操作的意味論	5
7	不変条件	6
第 III 部	証明	6
8	主定理	7
9	デッドロック自由の証明	7
10	公理を使っていないこと	8
第 IV 部	範囲	8
11	証明できる最大の構文	8
11.1	入れても保たれるもの	8
11.2	入れると壊れるもの	8
12	空虚でないことの確認	9
13	効果	9
13.1	型に載せる	9
13.2	規則の要は二つ	9
13.3	不変条件	10
13.4	定理	10
14	言えないこと	10
15	残っていること	11
15.1	効果は入れた	11
15.2	期限を足す	11
15.3	名前の整理	11
16	まとめ	11

第 I 部

位置づけ

1 三つの版の関係

版	日付	証明の形	デッドロック自由の言い方
第 1 版	2026-07-28	機械検証 (Coq)	<code>await</code> を言語から除いた断片についてのみ
第 2 版	2026-07-30	紙	期限を必須にして、有限時間の失敗に変換する
第 3 版 (本稿)	2026-08-23	機械検証 (Coq)	<code>await</code> を含んだまま、そもそも詰まらないことを言う。効果も機械検証に入れた

三つは重なっているが同じではない。違いは何によって詰まらないと言うかである。

- 第 1 版は待つ構文を捨てた。待たなければ詰まらない。
- 第 2 版は待ちに上限を置いた。上限があれば有限時間で終わる。ただし終わり方が失敗でもよい。
- 第 3 版は待ちの向きを揃えた。循環しなければ詰まらない。失敗に変換する必要が無い。

注意 1 (名前について). 第 2 版も、そこで切り出した部分言語を AIPL^{-2} と呼んでいる。本稿の AIPL^{-2} とは中身が違う — 第 2 版のそれは効果と期限を持ち、本稿のそれは義務レベルを持つ。名前が衝突しているのは、どちらも「第 1 版の次」を意味していたためである。

本稿では、機械検証された言語を AIPL^{-2} と呼ぶ。第 2 版の言語を区別して指す必要があるときは $\text{AIPL}^{-\varepsilon}$ と書く。

なお本稿の執筆中に効果を機械検証へ合流させた (§13)。残る隔たりは期限だけである。

2 何を証明したいのか

三つある。

性質	主張
型健全性	型の付いた構成が一步進んでも、型の付いた構成のままである (保存)。加えて、各アクターの状態と解決済み <code>future</code> の値は常に宣言された型を持つ
型安全性	型の付いた構成から到達できる構成は <code>stuck</code> でない。「終わってもいないのに一步も進めない」ことが起きない
デッドロック自由	型の付いた構成が <code>blocked</code> にならない。すなわち「メールボックスが空で、全タスクが待ち状態」という構成が現れない

三つ目が本稿の主題である。第 1 版はこれを、待てる構文を除いた断片についてしか言えなかった。

第 II 部

定式化

3 構文

型と項は次のとおりである。実装の `src/ast.ml` と `src/types.ml` のアクター核を取り出したものである。

$$\tau ::= \text{int} \mid \text{bool} \mid \text{unit} \mid \text{actor}[c] \mid \text{future}\langle\tau\rangle@n$$
$$\begin{aligned} e ::= & n \mid b \mid () \mid x \mid \text{self} \mid o_o \mid f_k \\ & \mid e + e \mid e < e \mid \text{if } e \text{ } e \text{ } e \mid \text{var } x = e; e \\ & \mid \text{get} \mid \text{set } e \mid \text{new } c \\ & \mid \text{future } e.m(e) \mid \text{await } e \\ & \mid e; e \mid \text{while } e \text{ do } e \end{aligned}$$

o_o と f_k は実行時にだけ現れる参照である。`future $\langle\tau\rangle@n$` が本稿の要で、「型 τ の値を、レベル n のメソッドが埋める future」を表す。

3.1 糖衣

次の三つは規則を増やさずに書ける。

書き方	展開
<code>now t.m(e)</code>	<code>await (future t.m(e))</code>
<code>send t.m(e);</code>	<code>(future t.m(e)); ()</code>
<code>e1; e2</code>	逐次実行（構文として持つ）

`while` は `if` へ展開する形にした。束縛を作らないので変数捕獲の心配が無い。

$$\text{while } c \text{ do } b \longrightarrow \text{if } c \text{ } (b; \text{while } c \text{ do } b) \text{ } ()$$

4 表と構成

プログラムは五つの表で与える。

表	内容
<code>stype(c)</code>	クラス c の状態の型
<code>sinit(c)</code>	クラス c の状態の初期値
<code>mtab(c, m)</code>	$c.m$ の引数型と返り値型
<code>body(c, m)</code>	$c.m$ の本体（引数は変数 0）
<code>lvl(c, m)</code>	$c.m$ の義務レベル

実行時の構成は (Ω, ms, ts) である。 Ω はヒープで、アクター表・状態表・future 表・future の値を持つ。future 表は (τ, n) の対を持つ — 型と、埋める側のレベルである。 ms はメールボックス（飛

んでいるメッセージの並び)、 ts は走っているタスクの並びである。タスクは (o, k, e) の三つ組で、「アクター o の上で式 e を評価し、その値で **future** k を埋める」を意味する。

5 型付け

判断は

$$\Omega; \Phi \vdash_{c,L} \Gamma \vdash e : \tau$$

と読む。 Ω はアクター表、 Φ は **future** 表、 c はいま検査しているメソッドが属するクラス、 L はそのメソッドの義務レベルである。 L を判断に持たせたのが第 1 版との違いである。

主要な規則だけ挙げる。

規則	内容
参照	$\Phi(k) = (\tau, n)$ ならば $f_k : \text{future}(\tau)@n$
送信	$e_0 : \text{actor}[c']$ かつ $\text{mtab}(c', m) = (\tau_a, \tau_r)$ かつ $e_1 : \tau_a$ ならば $\text{future } e_0.m(e_1) : \text{future}(\tau_r)@l\text{vl}(c', m)$
待ち	$e : \text{future}(\tau)@n$ かつ $L < n$ ならば $\text{await } e : \tau$

待ちは必ず上へ向かう。レベル L のメソッドが待てるのは、レベルが L より真に大きいメソッドが埋める **future** だけである。これがデッドロック自由の全部である。

プログラム全体が型検査を通っているとは、次を満たすことである。

仮定	内容
<code>sinit_value</code>	状態の初期値は値である
<code>sinit_ok</code>	状態の初期値は宣言された型を持つ
<code>bodies_ok</code>	$\text{mtab}(c, m) = (\tau_a, \tau_r)$ ならば、 $c.m$ の本体はレベル $\text{lvl}(c, m)$ のもとで τ_r の型を持つ

これらは Coq の Section の Hypothesis であり、End で各定理の前提に変わる。公理ではない。

6 操作的意味論

タスク一步の関係 $\Omega; o \vdash e \rightarrow \Omega'; \text{out}; e'$ と、構成一步の関係 $\Rightarrow$ の二段である。 out は新たに送出されるメッセージの並びである。

構成の一步は三つしかない。

規則	内容
CTask	タスクが一步進む。送出されたメッセージはメールボックスに入る
CFinish	タスクの式が値になった。その値が reply の値であり、担当していた future を埋めてタスクは消える
CDeliver	メッセージを配送し、メソッド本体を新しいタスクとして起こす

送信の規則が要点である。

$$\Omega; o \vdash \text{future } o_{o'}.m(v) \rightarrow \Omega[\Phi \dashv\vdash (\tau_r, \text{lvl}(c', m)); [(o', m, v, |\Phi|)]; f_{|\Phi|}]$$

すなわち **future** を一つ増やすのと同時に、それを埋めるためのメッセージを必ず一通出す。この対応が次節の不変条件になる。

7 不変条件

構成が正しいとは、次の五つを満たすことである。

条件	内容
heap_ok	表の長さが揃い、各アクターの状態は宣言された型を持ち、解決済み future の値は記録された型を持つ
msg_ok	飛んでいるメッセージの宛先は実在し、そのクラスはそのメソッドを持ち、引数の型が合い、 担当する future のレベルが $\text{lvl}(c, m)$ に一致する
task_ok	タスクは、 自分が埋める future のレベルのもとで型が付く
prod_ok	未解決の future には必ず「埋める者」がいる — メールボックスの中のメッセージか、走っているタスクのどちらかである
ext	起動時のアクター表は現在のアクター表に含まれる

prod_ok が第 3 版で足したものである。送信が **future** とメッセージを同時に作るので初期状態で成り立ち、配送はメッセージをタスクに変え（担い手に移る）、完了は **future** を埋める（義務が消える）ので、一歩ごとに保たれる。

第 III 部

証明

8 主定理

定理	主張
<code>no_method_not_understood</code>	型の付いた構成を飛ばすメッセージは、宛先が実在し、そのクラスがそのメソッドを持ち、引数と <code>future</code> の型もレベルも合っている
<code>preservation</code>	一歩進んでも <code>conf_ok</code> が保たれる
<code>preservation_star</code>	何歩進んでも保たれる
<code>type_safety</code>	到達できる構成は <code>stuck</code> でない
<code>deadlock_free</code>	型の付いた構成は <code>blocked</code> にならない
<code>progress_total</code>	終状態か、一歩進めるか、の二択
<code>deadlock_free_star</code>	到達できる構成すべてについて同じことを言う
<code>state_type_invariant</code>	どの時点でも各アクターの状態は宣言型を持つ
<code>future_type_invariant</code>	解決済み <code>future</code> の値は宣言された返り値型を持つ
<code>effect_no_increase</code>	一歩進んでも効果は増えない
<code>effect_soundness</code>	タスクの効果は宣言された効果に収まり続ける
<code>await_charges_callee</code>	待つと呼び先の効果を必ず引き継ぐ
<code>send_does_not_charge</code>	送るだけでは引き継がない

9 デッドロック自由の証明

定理 2 (デッドロック自由). `conf_ok(C)` ならば C は *blocked* でない。

Proof. $C = (\Omega, ms, ts)$ が *blocked* だと仮定する。定義より $ms = []$ 、 $ts \neq []$ 、そして ts のすべてのタスクが待ち状態である。

タスクの**レベル**を、そのタスクが埋める `future` に記録されているレベルと定める。 ts は空でない有限の並びなので、**レベルが最大のタスク** $t = (o, k, e)$ が取れる。

t も待ち状態である。待っている式は必ず未解決の `future` をひとつ名指ししており、`task_ok` よりタスクはレベル $L = \text{lvl}(k)$ のもとで型が付いているから、待ちの規則によりその `future` のレベル n は $L < n$ を満たす。

`prod_ok` より、その `future` には埋める者がいる。 $ms = []$ だからメッセージではありえず、したがってタスクである。そのタスクが埋める `future` のレベルは n であり、 $n > L$ である。これは t のレベル最大性に反する。□

系 3 (進行、二択). `conf_ok(C)` ならば、 C は終状態であるか、一歩進める。

Proof. 第 1 版の進行定理は三択 (終状態・一歩進める・*blocked*) である。上の定理が第三の枝を消す。□

所見 4. 証明に効いているのは二つだけである — **待ちの規則** ($L < n$) と、`prod_ok` (未解決の `future` には埋める者がいる) である。前者は型の側、後者は実行時の側にある。片方だけでは足りない。待ちの向きが揃っていても、埋める者が居なければ待ちは返らない。埋める者が居ても、向きが循環していれば互いを待つ。**型と実行時の不変条件が噛み合っ**て初めて言える性質である。

10 公理を使っていないこと

九つの定理すべてについて Print Assumptions を実行し、Closed under the global context を確認した。Makefile の check ターゲットがこれを自動で行う。

第 IV 部

範囲

11 証明できる最大の構文

三つの性質が同時に成り立つ範囲はどこまでか。機構ごとに調べた。

11.1 入れても保たれるもの

機構	理由
while / 逐次実行	止まらない繰り返しは「デッドロック」ではなく「停止しない」である。進行定理は一步進めることしか言っていないので、三つとも保たれる。while は if へ展開する形にしたので束縛を作らず、変数捕獲が起きない
動的なアクター生成	new はヒープを伸ばすだけである
future の入れ子	await の中に await があってもよい。レベルは式の位置ではなくメソッドに付くので、入れ子でも同じ条件で足りる

11.2 入れると壊れるもの

機構	壊れるもの
select	待つ相手が誰かは型に現れないので、レベルが効かない。進行の枝が一つ増え、デッドロック自由が壊れる。実装では期限の義務づけと送り手の存在検査で補っている
timeout	時間を意味論に入れる必要がある。進行は保てるが、主張が「詰まらない」から「詰まっても諦める」に変わる — 第 2 版がまさにこれである
any / 多重定義 / 多相	標準形補題が壊れる。値の形が型から決まらないので進行が通らない
become	受け手のクラスが実行時に変わると、配送時に型が合っている保証が崩れる (no_method_not_understood が壊れる)。なお実装では become 自体を言語から外した
他アクターの状態への直接代入	heap_ok の「各アクターの状態は宣言型を持つ」が、自分のメソッド内でしか保てなくなる

所見 5. この線引きは、言語の設計として機構を採るか落とすかを決めた判断とは独立に引いたのに、ほぼ同じところに落ちた。become も any も多相も、設計として落とした理由と、証明が通らない理由が一致している。証明の通る範囲は、設計判断の後追いではなく、その裏づけであった。

12 空虚でないことの確認

定理はすべて「プログラムが型検査を通っている」という仮定のもとにある。レベルの条件が満たせないなら定理は空虚なので、確かめた。

```
class Svc    { method get(x) : int { reply(x); } }           // レベル 1
class Caller { method run(x) : int {                       // レベル 0
  while (x < 0) { }                                         // 第3版で広げた構文
  reply(now svc.get(x));                                   // await を含む
} }
```

このプログラムについて、`sinit_value · sinit_ok · bodies_ok` をすべて Coq で証明し、初期構成が `conf_ok` であることを示した。したがってデッドロック自由が適用でき、進行も二択で言える。

対照実験も行った。`lvl(Svc, get)` を 1 から 0 に変えると（すなわち待ちの向きを平らにすると）、`bodies_ok` の証明が通らなくなる — $0 < 0$ が示せないからである。条件が実際に効いている。

13 効果

レベルを載せたのと同じやり方で効果も載せた。

13.1 型に載せる

$$\text{future}\langle\tau\rangle@n \longrightarrow \text{future}\langle\tau\rangle@n!\varepsilon$$

判断も一つ増える。

$$\Omega; \Phi \vdash_{c,L} \Gamma \vdash e : \tau!\varepsilon$$

ε は効果名の集合である（形式化では `list nat`、包含は `incl`、合併は `++`）。

13.2 規則の要は二つ

規則	効果
送信	<code>future e₀.m(e₁)</code> の効果は $\varepsilon_0 \cup \varepsilon_1$ である — 呼び先の効果を引き継がない。待たないからである。ただし結果の型には <code>eff(c', m)</code> を記録する
待ち	<code>await e</code> の効果は $\varepsilon_e \cup \varepsilon_c$ である — 呼び先の効果を引き継ぐ。 ε_c は <code>future</code> の型に記録されていたものである

これは実装の挙動と同じである。実装でも `send` は伝播せず、`await` が伝播する。プログラム全体の仮定にも一つ足す — `bodies_ok` は「本体の効果が宣言した効果 `eff(c, m)` に収まる」を要求する。実装が推論した効果を注釈と照合するのと同じ形である。

13.3 不変条件

`task_ok` に一項足す — タスクの効果は、そのタスクが埋める `future` に記録された効果に収まる。配送のときに `bodies_ok` が入り、以後は保存で保たれる。

13.4 定理

定理	主張
<code>effect_no_increase</code>	一歩進んでも効果は増えない
<code>effect_soundness</code>	走っているタスクの効果は、担当する <code>future</code> に記録された効果に収まり続ける
<code>await_charges_callee</code>	待つと呼び先の効果を必ず引き継ぐ
<code>send_does_not_charge</code>	送るだけでは引き継がない

`effect_soundness` の中身は次のとおりである — メソッドが宣言した効果は、そのメソッドが実際に行うことを (`now` で呼んだ先も含めて) 覆っている。

系 6 (一段隔てても隠せない). $\text{eff}(c, m)$ に `ai` が入っていないければ、 $c.m$ の本体は `ai` を持つメソッドを `await` できない。

Proof. 待ちの規則より、`await` の効果は呼び先の効果を含む。 `bodies_ok` より本体の効果は $\text{eff}(c, m)$ に収まる。包含の推移性による。 $\square$

これが実装の動機そのものである — 実時間で動くアクターが、機外のモデルを同期で待てない。

所見 7. 効果を足すのに、レベルのときと同じ形が使えた — `future` の型に載せ、待ちで課金する。違いは「レベルは比較し、効果は合併する」ことだけである。どちらも「呼び先について知りたいことを、`future` の型に持たせる」という一つの考え方の例である。

14 言えないこと

- **停止性**. デッドロック自由は「詰まらない」であって「必ず終わる」ではない。 `while 1 > 0 do {}` は止まらないが、それは待ちではないので進行定理と矛盾しない。
- **飢餓**. あるタスクが永久に選ばれない可能性は排除していない。進行定理は「どれかが進める」しか言わない。
- **宛先が動的な場合**. レベルは `mtab` から引くので、宛先のクラスが静的に決まらなければ効かない。
- **同じクラスの別インスタンス**. レベルはメソッド単位なので、親が子を待つ木構造のような場合も一律に扱われる。緩めるにはレベル変数が必要。
- **期限**. 本稿の形式化には入っていない (次節)。効果は入れた (§13)。

15 残っていること

15.1 効果は入れた

本稿の執筆中に足した (§13)。第 2 版が紙で示していた効果の健全性が、機械検証になった。

15.2 期限を足す

期限を入れると時間を意味論に持ち込む必要がある。得られるのは「詰まっても有限時間で失敗する」であって、本稿の「詰まらない」より弱い。両方を持つ形（レベルで循環を止め、期限で残りを救う）が実装の姿なので、形式化としてもそこが終着点になる。

15.3 名前の整理

効果が入ったので、第 2 版の $\text{AIPL}^{-\varepsilon}$ と本稿の AIPL^{-2} の違いは**期限だけ**になった。期限を足した時点で名前は一本化できる。それまでは本稿の記法（ AIPL^{-2} は機械検証された言語、 $\text{AIPL}^{-\varepsilon}$ は第 2 版の言語）で区別する。

16 まとめ

第 1 版はデッドロック自由を、`await` を言語から取り除いた断片についてしか言えなかった。第 2 版は期限で有限時間の失敗に変換した。本稿は**待ちを取り除かず、期限にも頼らず**、デッドロック自由そのものを機械検証した。

足したのは四つだけである — `future` の型にレベルを持たせ、判断にレベルを足し、待ちに $L < n$ を要求し、「未解決の `future` には埋める者がいる」を不変条件に置いた。証明の骨は「レベルが最大のタスクを取る」ことである。

そして三つの性質が同時に成り立つ最大の構文を調べたところ、その境界は**言語の設計判断と独立に引いたのに、ほぼ同じところに落ちた**。証明の通る範囲は、設計の後追いではなく、その裏づけであった。

再現方法

```
cd ~/aios/abclcp/coq
make          # AIPLSoundness2.v と AIPLSoundness2Example.v を含めてビルド
make check    # 主定理の Print Assumptions を確認
```

ファイル	内容
coq/AIPLSoundness2.v	本稿の形式化と証明 (1,314 行)
coq/AIPLSoundness2Example.v	空虚でないことの確認 (123 行)
coq/AIPLSoundness.v	第 1 版 (1,121 行)
docs/aipl_soundness2_ja.pdf	第 2 版 (紙の証明。効果と期限)
docs/aipl_paper5_ja.pdf	論文。言語全体の設計と採否